



An ensemble learning algorithm based on multimodal differential evolution and extreme learning machine

Jie Ma¹ · Hong Li¹ · Weifeng Gao¹ · Jin Xie¹

Received: 28 September 2024 / Accepted: 12 April 2025
 © The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract

The neural network ensemble (NNE) is a widely recognized and effective classification method. It is imperative to concurrently optimize the topological structures and weight parameters of all base neural networks (NNs) within the NNE. This simultaneous optimization is crucial for balancing the accuracy and diversity of the base NNs, leading to superior classification performance. The extreme learning machine (ELM) is a rapid non-iterative learning strategy that has garnered significant attention recently due to its robust generalization capabilities, though effective only for fixed-structure NNs. Hence, to tackle these intricate challenges, this paper introduces a novel ensemble learning algorithm based on a multimodal differential evolution (MMDE) and ELM, called MMDE-ELM-NNE. The proposed algorithm optimizes both the structures and weight parameters of all the base NNs within an NNE in one run. MMDE-ELM-NNE employs a unique coding scheme to optimize the structures and input parameters of the base NNs, while the ELM's output parameters are computed directly. Furthermore, the MMDE algorithm has been enhanced better to meet the high-dimensional optimization demands of NNE tasks, ultimately enhancing classification accuracy. The experiments on twenty multimodal optimization problems are carried out and compared with eleven existing algorithms to verify the effectiveness of the proposed MMDE. Additionally, the experimental results on classification benchmark datasets relying on a Friedman test demonstrate that MMDE-ELM-NNE significantly outperforms ensembles of MMDE or ELM in terms of generalization prowess. Furthermore, the proposed algorithm showcases competitive capabilities when compared against other state-of-the-art classification algorithms.

Keywords Neural network ensemble · Ensemble learning · Differential evolution · Multimodal optimization · Extreme learning machine · Classification

List of symbols

$\vec{t}_i$	Individual i (neural network structure and parameters)	$mean_A(\cdot)$	Arithmetic mean
$\vec{v}_i$	Mutant vector of individual i	$\vec{w}_i$	Input weight vector of hidden node i
$\vec{u}_i$	Trial vector of individual i	b_i	Bias of hidden node i
$\vec{t}_{best}$	Best individual in the population	$\vec{\beta}_i$	Output weight vector of hidden node i
F_i	Scaling factor of individual i	$\vec{o}_i$	Output value of the network
CR_i	Crossover rate of individual i	$g(\cdot)$	Activation function (sigmoid function by convention)
μ_F	Mean of successful F_i values	$\mathbf{H}$	Output matrix of the hidden layer
μ_{CR}	Mean of successful CR_i values	β	Output weight matrix
S_F	Set of successful F_i	$\mathbf{Y}$	Desired output matrix
S_{CR}	Set of successful CR_i	$\mathbf{H}^\dagger$	Moore-Penrose pseudo-inverse of $\mathbf{H}$
b	Constant, typically set to 0.1	δ	State parameters of hidden layer neurons
$mean_L(\cdot)$	Lehmer mean	s_i	Activation status of neuron i in the hidden layer
		D	Dimension of the solution space
		N	Number of training samples
		m	Number of classes
		y_{ij}	Actual output of class j for sample i

✉ Hong Li
 lihong@mail.xidian.edu.cn

¹ School of Mathematics and Statistics, Xidian University, Xi'an 710071, Shaanxi, China

o_{ij}	Desired output of class j for sample i
ϵ_0	Small given number for subpopulation stagnation
θ	Threshold for subpopulation convergence
St	Integer for stagnation tolerance
η	Count of generations with no update to the best individual

1 Introduction

In practical scenarios, there exist optimization problems that require the identification of multiple global optimal solutions (GOSs), a characteristic known as multimodal optimization problems (MMOPs). Over the past two decades, MMOPs have attracted considerable attention and research interest. In tackling MMOPs, evolutionary algorithm (EA), a population-based metaheuristic algorithm, has emerged as an effective tool; however, they often fall short in capturing the nuances of problem-specific knowledge and local search heuristics. Conventional EAs are inappropriate for MMOPs as they are typically designed to find only one GOS in an optimization process.

Against this backdrop, the field of memetic computing emerges as a potent paradigm. Memetic computing, which integrates the principles of evolutionary computation with local search methods, has demonstrated its efficacy in enhancing the performance of optimization algorithms. This approach is inspired by the cultural evolution of memes, where GOSs or 'memes' are replicated and refined within a population, leading to the emergence of more effective strategies. Thus, conventional EAs have been augmented with niching techniques to bolster population diversity and explore a wider range of GOSs. Various methods, including crowding [49], clustering [58], speciation [32], locality sensitive hashing [15], fitness sharing [9], and clearing [41], etc., have been amalgamated with different types of EAs to tackle MMOPs [23, 35, 56, 57]. Among these methods, integrating niching techniques into differential evolutions (DEs) for solving MMOPs, known as multimodal DEs (MMDEs), stands out for their rapid convergence rate, straightforward implementation, and robust performance even in the presence of noisy or incomplete data.

Training multiple base neural networks (NNs) in a single run can be considered an MMOP, where the structure and parameters of an NN are regarded as the variables in an optimization problem, with the objective of minimizing the error between the actual and desired NN outputs. Hence, the practical approach of MMDEs could also be harnessed for NNE optimization. Compared to simply aggregating a specific number of base NNs trained by conventional EAs or backpropagation, which can be interpreted as addressing several single-modal optimization problems, MMDEs

offer a significantly more efficient method for training NNEs by considering them an MMOP. This is primarily because conventional EAs and backpropagation can obtain only one optimal base network at a time, requiring several repetitions to train multiple NNs for the NNE, which is time-consuming and inevitably results in redundant or homogeneous NNs. Training NNEs using MMDEs, on the other hand, simultaneously generates multiple diverse NNs in a single run by partitioning the entire population into multiple subpopulations, each tasked with seeking one GOS. Therefore, using MMDEs can significantly facilitate NNE training and conserve computational resources.

In recent years, one of the notable additions to MMDEs is the innovative approach known as adaptive niching DE (ANDE) [22]. This algorithm effectively navigates the exploration and exploitation to uncover a diverse range of accurate GOSs for MMOPs. However, when applied to optimize an NNE, ANDE's efficacy is constrained when identifying multiple base NNs that balance both diversity and classification accuracy, with specific components failing to operate as intended. This discrepancy may stem from the substantial disparities between discovering GOSs of a function in a test suite and optimizing NNs within an NNE. This topic will be elaborated in Sect. 3. Considering both ANDE's superiority in MMOPs and the unique demands of NNE optimization tasks, this study leverages ANDE as the baseline model, upon which several significant improvements are proposed to meet the NNE optimization requirements. The significant contributions of this study are as follows:

1. An improved heuristic clustering method tailored for high-dimensional data;
2. A refined archive selection scheme designed to identify diverse NNs efficiently with minimal computational resources;
3. A redesigned subpopulation adaptation strategy that effectively increases the population variety while relying on fewer computer resources.

With the enhanced MMDE algorithm, optimizing multiple NNs concurrently with accuracy and diversity becomes feasible. In contrast to conventional backpropagation NNs, this study adopts the non-iterative extreme learning machine (ELM) algorithm. ELM is a machine learning algorithm that has gained popularity in recent years. This algorithm randomly assigns input weights and analytically calculates the output weights, which leads to accelerated learning and makes ELM less prone to overfitting and allows for efficient training on high-dimensional data. This incorporation of the ELM into the proposed algorithm thus confers several advantages, including a rapid learning pace, a straightforward

ward implementation, and the capability to handle large-scale datasets adeptly.

Despite all the advantages associated with ELM, the absence of a systematic approach for determining the optimal number of hidden neurons poses a challenge for ELM [2]. Additionally, the inherent randomness in ELM may fluctuate learning performance. Therefore, this paper designed a novel encoding strategy to optimize the ELM's weight parameters and the number of hidden neurons simultaneously, while the ensemble strategy effectively mitigates the impact of randomness. This culminates in the proposal of an NNE optimization algorithm based on MMDE and ELM, named as MMDE-ELM-NNE. The combination of MMDE and ELM in the MMDE-ELM-NNE algorithm is synergistic. MMDE's ability to identify multiple optimal structures provides the diversity required for an effective ensemble, while ELM's rapid learning capabilities ensure that each structure is optimized for accuracy. Together, they offer a powerful approach to neural network ensemble optimization, addressing the challenges of structure and weight optimization in a unified framework. Extensive experiments are conducted on benchmark datasets for classification problems to validate the feasibility and effectiveness of MMDE-ELM-NNE. Experimental results demonstrate the proposed MMDE-ELM-NNE's superior competitiveness compared to MMDE, ELM, and other metaheuristic algorithms.

This study advances neural network ensemble optimization with four pivotal aspects:

1. **Innovations in MMDE:** We introduce an improved MMDE framework featuring a high-dimensional clustering method, an efficient archive selection scheme for diverse NNs, and a subpopulation adaptation strategy that boosts diversity using minimal computational resources.
2. **NNE transformation as an MMOP:** By framing the neural network ensemble as a multimodal optimization problem (MMOP), we enable the use of advanced MMDE techniques to concurrently optimize multiple base neural networks, identifying multiple global optima in a single run.
3. **Integration of ELM over backpropagation:** We integrate the ELM algorithm into the MMDE process, leveraging its non-iterative learning to expedite training and enhance classification accuracy, offering a more efficient alternative to traditional backpropagation.
4. **Simultaneous Optimization Encoding:** Addressing the challenge of optimal neural network structure determination, we propose a novel encoding strategy that optimizes ELM's weight parameters and hidden neurons simultaneously, streamlining ensemble development by mitigating ELM's inherent randomness.

The remainder of this paper is structured as follows. Section 2 overviews DE, MMDEs, and ELM. Section 3 outlines the specifics of the proposed enhanced MMDE algorithm. Section 4 introduces the motivation and the workflow of the proposed MMDE-ELM-NNE algorithm. Section 5 presents the results of the experiments and their analysis, with concluding remarks and future work discussed in Sect. 6.

2 Background

This section offers a comprehensive overview of the key operations in DE and explores the adaptive parameter mechanisms such as JADE. It also introduces various categories of MMDEs and elucidates the principles underlying the ELM. Furthermore, the section offers a comprehensive examination of Network Architecture Search and Hyperparameter Optimization, outlining their significance and interplay in the realm of machine learning.

2.1 Differential evolution

DE is a popular real-number stochastic searching algorithm proposed by Storn and Price in 1995 [47]. There are three fundamental operations in DE: mutation, crossover, and selection, which are illustrated below. DE repeatedly performs the three steps to iteratively generate new populations until the termination condition is met.

The performance of DE hinges significantly on the configuration of two crucial control parameters: the scaling factor F and the crossover rate CR . However, determining optimal values for these parameters under different problem landscapes is a challenging task. To address this issue, several adaptive parameter mechanisms have been proposed for DE, as discussed in prior studies [6, 26, 34, 43]. These mechanisms involve adjusting the control parameters dynamically based on feedback from the search process. Among these approaches, JADE [26] stands out for its superior performance and is selected as the parameter updating method in this article. In contrast to traditional DE, JADE assigns each individual $\vec{t}_i$ with its own scaling factor F_i and crossover rate CR_i . In each generation, F_i is updated by a Cauchy distribution $F_i \sim \text{Cauchy}(\mu_F, 0.1)$ and CR_i is updated according to a normal distribution $CR_i \sim N(\mu_{CR}, 0.1)$. Initialized at 0.5, the respective mean value μ_F and μ_{CR} are updated every generation as follows:

$$\mu_F = (1 - b) \cdot \mu_F + b \cdot \text{mean}_L(S_F), \quad (1)$$

$$\mu_{CR} = (1 - b) \cdot \mu_{CR} + b \cdot \text{mean}_A(S_{CR}), \quad (2)$$

where b is a constant that is typically set to 0.1; S_F and S_{CR} denote the sets of all F_i and CR_i that successfully enhance

the trial vectors, respectively; $mean_L(\cdot)$ and $mean_A(\cdot)$ are the Lehmer mean and arithmetic mean, respectively.

2.1.1 Mutation

In each generation, a mutant vector $\vec{v}_i$ is generated for each individual $\vec{t}_i$ by adding scalable differences between other individuals to the base individuals. In this paper, the DE/best/1 strategy [46] is adopted to generate the mutant vector:

$$\vec{v}_i = \vec{t}_{best} + F_i \cdot (\vec{t}_{r1} - \vec{t}_{r2}), \quad (3)$$

where $\vec{t}_{best}$ is the best individual of the population with the highest fitness value, and $\vec{t}_{r1}$ and $\vec{t}_{r2}$ are distinct individuals randomly selected from the current population, which are both different from the individual $\vec{t}_i$.

2.1.2 Crossover

With the generated mutation vector $\vec{v}_i$, the current vector $\vec{t}_i$ is combined with it to get a new trial vector $\vec{u}_i$, which is given by:

$$u_{i,j} = \begin{cases} v_{i,j}, & \text{if } rand(0, 1) \leq CR_i, \text{ or } j = I \\ t_{i,j}, & \text{otherwise} \end{cases}, \quad (4)$$

where CR_i denotes the crossover rate that establishes how distinct the trial vector $\vec{u}_i$ is from the current vector $\vec{t}_i$; $rand(0, 1)$ is a random number between (0, 1), I is a random integer generated from $[1, D]$, and the formula $j = I$ makes sure that at least one element of the current vector $\vec{t}_i$ mutates.

2.1.3 Selection

During the selection stage, a comparison is made between the fitness values of the trial vector $\vec{u}_i$ and the current vector $\vec{t}_i$ to generate offspring using elitist tactics. In the case of a maximization optimization problem, the individual with the superior fitness value will be designated as the offspring of the next generation. Notably, superior control parameter values have a tendency to produce individuals with a higher likelihood of survival, these values can therefore be maintained and further propagated to more offspring. The selection vector is given by:

$$\vec{t}_i = \begin{cases} \vec{u}_i, & \text{if } f(\vec{u}_i) > f(\vec{t}_i) \\ \vec{t}_i, & \text{otherwise} \end{cases}, \quad (5)$$

where $f(\cdot)$ is the objective function to be optimized.

2.2 Multimodal differential evolution

A number of DE variants have been developed to tackle MMOPs, typically classified into niching technique-based methods and multi-objective optimization-based methods. Niching technique-based methods, as opposed to multi-objective optimization-based methods, are simpler to implement and demand fewer computational resources, rendering them more suitable for MMOPs. These techniques, inspired by biological principles, aim to explore diverse GOSs by segregating the population into multiple niches. Notable examples of niching DE algorithms include Crowding DE (CDE) [49] and Speciation DE (SDE) [32]. However, these algorithms involve intricate niching parameters like crowd size and species radius. To address this challenge, Qu et al. introduced Neighborhood-Based Crowding DE (NCDE) and Neighborhood-Based Speciation DE (NSDE), which conduct mutation within each Euclidean neighborhood to preserve diverse GOSs [44]. Expanding on this concept, Gao et al. employed a clustering technique to divide subpopulations, resulting in Self_CCDE and Self_CSDE [14]. Zhang et al. proposed a rapid niching method using locality-sensitive hashing to circumvent pairwise distance computations [62].

In recent years, novel multimodal niching DE strategies have emerged, such as the local information-based mutation strategy in LoICDE and LoISDE by Biswas et al. [5], and the ensemble neighborhood mutation strategy in SDE by Hui et al. [23]. Additionally, Wang et al. introduced MMDEss, incorporating a contour prediction approach [52], adaptive estimation distribution [53], and fitness and distance-based local search [54] into DE. Wei et al. employed a penalty strategy with a dynamic penalty radius to devise Penalty-based Multimodal Optimization DE (PMODE) [55]. Jiang et al. treated the niche center distinguish (NCD) problem as an optimization problem, proposing NCD-based DE (NCD-DE) [25]. The research on MMDEs has made significant strides, enhancing their efficiency in solving MMOPs.

2.3 Extreme learning machine

It is known that traditional feedforward NNs often exhibit slow learning rates due to the iterative gradient-based learning algorithm they employ. To address this limitation, Huang [19–21] proposed a learning algorithm called ELM for training single-hidden layer feedforward neural networks (SLFNs). ELM adopts a unique approach by randomly selecting hidden nodes and analytically computing the output weights of SLFNs. This method delivers exceptional generalization performance at an incredibly rapid learning rate, often thousands of times faster than conventional feedforward NNs [19].

Given a training dataset containing of N arbitrary distinct samples $(\vec{x}_i, \vec{y}_i)_{i=1}^N$, where $\vec{x}_i = [x_{i1}, x_{i2}, \dots, x_{in}]^T \in \mathbf{R}^n$

consists of n input attributes and $\vec{y}_i = [y_{i1}, y_{i2}, \dots, y_{im}]^T \in \mathbf{R}^m$ consists of m output labels. The output of a standard SLFN with L hidden nodes can be calculated as:

$$\vec{o}_j = \sum_{i=1}^L \vec{\beta}_i g(\vec{w}_i^T \vec{x}_j + b_i), \quad j = 1, \dots, N, \quad (6)$$

where $\vec{w}_i = [w_{i1}, w_{i2}, \dots, w_{in}]^T \in \mathbf{R}^n$, $i = 1, 2, \dots, L$ represents the input weight vector between the n input nodes and the i th hidden node; b_i is the bias for the i th hidden node; $\vec{\beta}_i = [\beta_{i1}, \beta_{i2}, \dots, \beta_{im}]^T \in \mathbf{R}^m$ is the output weight vector between the i th hidden node and the m output nodes; $\vec{o}_i = [o_{i1}, o_{i2}, \dots, o_{im}]^T \in \mathbf{R}^m$ is the output value of the network; $g(\cdot)$ is the activation function and in this article the sigmoid function is selected by convention:

$$g(\vec{w}_i, \vec{x}_j, b_i) = \frac{1}{1 + \exp[-(\vec{w}_i \cdot \vec{x}_j + b_i)]}. \quad (7)$$

With suitable values of β , the L2 distance between the actual and desired outputs $\sum_{j=1}^N \|\vec{o}_j - \vec{y}_j\|$ can reach zero with probability one (proved in [21]). In this case, the Eq. (6) can be transformed into

$$\vec{y}_j = \sum_{i=1}^L \vec{\beta}_i g(\vec{w}_i^T \vec{x}_j + b_i), \quad j = 1, \dots, N. \quad (8)$$

Therefore, it is possible to combine these N equations and write them in a matrix form

$$\mathbf{H}\beta = \mathbf{Y}, \quad (9)$$

where

$$\mathbf{H} = \begin{pmatrix} g(\vec{w}_1, \vec{x}_1, b_1) & \dots & g(\vec{w}_L, \vec{x}_1, b_L) \\ \vdots & \ddots & \vdots \\ g(\vec{w}_1, \vec{x}_N, b_1) & \dots & g(\vec{w}_L, \vec{x}_N, b_L) \end{pmatrix}_{N \times L}, \quad (10)$$

$$\beta = \begin{pmatrix} \vec{\beta}_1^T \\ \vdots \\ \vec{\beta}_L^T \end{pmatrix}_{L \times m}, \quad (11)$$

$$\mathbf{Y} = \begin{pmatrix} \vec{y}_1^T \\ \vdots \\ \vec{y}_N^T \end{pmatrix}_{N \times m}. \quad (12)$$

The matrix $\mathbf{H}$ is the output matrix of the hidden layer; $\mathbf{Y}$ is the desired output matrix; β is the output weight matrix that can be computed by solving the linear least squares problem

$$\beta = \mathbf{H}^\dagger \mathbf{Y}, \quad (13)$$

where $\mathbf{H}^\dagger$ is the Moore-Penrose (MP) generalized inverse of $\mathbf{H}$. The steps involved in ELM training are shown in Algorithm 1.

Algorithm 1 Extreme learning machine (ELM)

Input: A training dataset $\{(\vec{x}_j, \vec{y}_j) | \vec{x}_j \in \mathbf{R}^n, \vec{y}_j \in \mathbf{R}^m\}_{j=1}^N$, an activation function $g(\cdot)$, and the number of hidden nodes L .

Output: The output weights β .

- 1: Initialize hidden biases and input weights randomly in the range $[-1, 1]$.
 - 2: Compute the hidden layer output matrix $\mathbf{H}$ using Eq. (10).
 - 3: Calculate the output weights $\beta = \mathbf{H}^\dagger \mathbf{Y}$ using the MP generalized inverse.
-

2.4 Network architecture search and hyperparameter optimization

Neural network architecture search (NAS) and parameter optimization are pivotal components in the realm of neural network (NN) design, significantly impacting the model's learning capabilities and complexity. The structural configuration of NNs, encompassing depth (number of layers) and width (number of neurons per layer), is crucial as it directly influences the model's capacity to learn and its overall complexity. Varying combinations of depth and width can lead to substantial performance disparities. Therefore, identifying the optimal architecture is essential for enhancing model performance. The quest for the best architecture involves exploring the vast search space of possible configurations to find the one that best fits the given dataset and task, balancing between underfitting and overfitting.

Parameter optimization is another critical research domain within NNs. It encompasses the initialization of weights and biases, selection of activation functions, and adjustment of learning rates. Optimization algorithms such as Stochastic Gradient Descent (SGD), Root Mean Square Propagation (RMSProp) [50], and Adaptive Moment Estimation (Adam) [30] are employed to update network weights, aiming to minimize the loss function. To prevent overfitting, regularization techniques such as L1 and L2 regularization, along with Dropout, are commonly utilized. These methods help in generalizing the model by reducing the complexity of the learned representations and preventing the model from fitting the noise in the training data.

As research in NNs deepens, a multitude of advanced methods for structure and parameter optimization has emerged. For instance, EAs are used for architecture search, simulating the process of natural selection to optimize network structures [1, 8, 10, 38]. These evolutionary approaches iteratively refine the architecture by applying genetic operators such as selection, crossover, and mutation, leading to the evolution of more efficient and effective NN structures.

Furthermore, Multi-Objective Genetic Algorithms (MOGAs) [40] have been proposed to find the optimal set of structural parameters [3, 7, 39, 42]. These methods address the multifaceted nature of NN optimization problems, where objectives such as accuracy, complexity, and generalization must be balanced. MOGA approaches provide a set of Pareto-optimal solutions, allowing researchers to select the best architecture based on the trade-offs between different objectives.

In conclusion, the fields of architecture search and parameter optimization are evolving rapidly, with researchers continually developing innovative methods to automate and enhance the design process of NNs. As the complexity of NNs increases, so does the need for sophisticated optimization techniques to navigate the intricate landscape of possibilities and uncover the most effective models for a given task.

3 An enhanced MMDE algorithm for NNE optimization

This section provides a detailed overview of the enhanced MMDE algorithm and its components tailored for optimizing NNEs.

3.1 Enhanced MMDE integration and its components

This subsection details the integration of the Enhanced MMDE algorithm, its innovative strategies, and the four key enhancements made to optimize neural network ensembles effectively, addressing the unique challenges of high-dimensional NNE optimization.

3.1.1 Introduction to ANDE and Its strategies

Numerous sophisticated methods have been developed to address MMOPs, with ANDE [22] standing out as a particularly effective approach. ANDE introduces three innovative strategies:

1. A heuristic clustering method that adaptively identifies non-overlapping subpopulations based on the overall population distribution in the specific landscape of the issue;
2. A dynamic parameter adaptation mechanism that enhances diversity within stagnant populations and reduces computational overhead by eliminating overlapping subpopulations;
3. An auxiliary movement scheme that aids stagnating subpopulations in escaping local optima and guides converging populations towards more promising regions.

These novel techniques synergistically empower ANDE with a high level of optimization capability for MMOPs. Huang's experiments on 20 multimodal test functions from the CEC2013 [33] benchmark suite demonstrate that ANDE outperforms most comparable niching algorithms across various evaluation protocols, including peak ratio, success rate, and convergence speed. Moreover, an analysis of the efficacy of the three components reveals that while each plays a distinct role within ANDE, they collectively contribute to its potent optimization performance.

3.1.2 Challenges in NNE optimization

Despite these advantages, the application of ANDE to NNE optimization does not exhibit the same level of performance as observed in test functions. This disparity may be attributed to the inherent differences between NNE optimization and MMOPs:

1. NNE optimization, although categorized as an MMOP for convenience, differs significantly in problem dimensions. While most functions in the CEC2013 test suite are 1-D to 3-D, NNE optimization problems can span hundreds of dimensions.
2. The objectives of optimization also diverge between the two problem types. MMOP evaluation protocols typically focus on maximizing the size of global optima within limited fitness evaluations, whereas NNE optimization emphasizes discovering a suitable number of NNs with high diversity and accuracy rather than the integrity of the global optimal solution space.

3.1.3 Enhancements in MMDE for NNE

To address these distinctions illustrated above, an enhanced MMDE is proposed to better cater to the requirements of NNE optimization. This enhanced algorithm incorporates four key enhancements:

1. The heuristic clustering method is improved by modifying the distance calculation criterion between two subpopulations, resulting in a more coherent partition of the population. This improvement proves especially advantageous in tackling high-dimensional optimization problems such as NNEs, offering a stronger foundation for the subsequent evolutionary progression of niches.
2. An archive selection strategy is proposed to identify the most promising non-overlapping solutions, striking a balance between the diversity and accuracy of the promising NNs saved in an archive.
3. An improved subpopulation adaptation strategy is proposed by eliminating its exclusion component and refin-

ing its augmentation component. In terms of exclusion, this alteration targets the limited effectiveness of the exclusion strategy in high-dimensional scenarios as observed during experimentation. Insights highlight the challenges faced by the exclusion strategy in efficiently eliminating unpromising subpopulations, leading to a substantial portion-over 40 percent-of the whole evolutionary process time being allocated to distance calculations. On another front, the modifications to the key algorithm in the augmentation stage focus on enhancing diversity within the population. The redesigned strategy effectively boost the diversity in stagnant populations while requiring fewer computational resources.

4. In addition, to better accommodate the distinctions of NNE optimization problems, specific threshold values and boundary conditions are adjusted.

3.2 Improved heuristic clustering method

Clustering serves as the predominant niching technique in DE. However, existing cluster-based niching algorithms commonly partition the entire population into a set of subpopulations, with the number of subpopulations typically predetermined by a hyperparameter known as the cluster size. Establishing an optimal cluster size poses a challenge as it is contingent upon the specific task at hand. To mitigate this challenge, a heuristic clustering approach was introduced, aiming to generate nonoverlapping subpopulations without introducing any hyperparameters [22]. This method adopts a strategy where each individual within the population is initially treated as a distinct subpopulation. Progressively, pairs of subpopulations are merged based on proximity until a termination condition is met. Finally, subpopulations containing fewer than four individuals are removed, as DE necessitates a minimum of four individuals per subpopulation for effective operation.

While this clustering method could effectively segments a population into nonoverlapping subpopulations in low-dimensional scenarios, challenges arise when it comes to high-dimensional problem spaces like optimizing the structures and weights of an NNE, which contains hundreds of dimensions. In such cases, the original heuristic clustering approach tends to amalgamate a significant portion of the population into a single subpopulation, which deviates from the goal of niching technology and provides a poor foundation for subsequent evolution process. The discrepancy in performance can be attributed to the distance metric employed in the original method, where the distance between two subpopulations (denoted as $D(\mathbf{P}_k, \mathbf{P}_s)$) is defined as the minimum distance between any two point from each set:

$$D(\mathbf{P}_k, \mathbf{P}_s) = \min_{\vec{t}_i \in \mathbf{P}_k, \vec{t}_r \in \mathbf{P}_s} d(\vec{t}_i, \vec{t}_r), \quad (14)$$

where $d(\vec{t}_i, \vec{t}_r)$ is the Euclidean distance between individuals $\vec{t}_i$ and $\vec{t}_r$.

This calculation criterion focuses solely on the closest point from a given subpopulation to any point in another subpopulation. Consequently, a subpopulation comprising more individuals naturally increases the likelihood of establishing shorter distances to other subpopulations, facilitating a higher tendency to assimilate additional individuals. Therefore, a subpopulation that incorporates more individuals at the beginning of the clustering process is more likely to gradually experience substantial expansion by nature. As a result, as is subsequently tested in Sect. 5.4.3, in high-dimensional problem spaces (exceeding 10 dimensions), the largest subpopulation could potentially encompasses nearly half of the entire population at the end of the clustering.

In the improved clustering method (shown in Algorithm 2), the measurement of the distance between two subpopulations $\mathbf{P}_k$ and $\mathbf{P}_s$ is defined as the average distance between any point in set $\mathbf{P}_k$ and any point in set $\mathbf{P}_s$:

$$D(\mathbf{P}_k, \mathbf{P}_s) = \frac{1}{|\mathbf{P}_k| |\mathbf{P}_s|} \sum_{\vec{t}_i \in \mathbf{P}_k, \vec{t}_r \in \mathbf{P}_s} d(\vec{t}_i, \vec{t}_r), \quad (15)$$

where $d(\vec{t}_i, \vec{t}_r)$ is the Euclidean distance between individuals $\vec{t}_i$ and $\vec{t}_r$, $|\mathbf{P}_k|$ and $|\mathbf{P}_s|$ are the numbers of individuals of subpopulation $\mathbf{P}_k$ and $\mathbf{P}_s$.

Algorithm 2 Improved heuristic clustering

Input: Initial population $\mathbf{P}_0$.

Output: Subpopulations $\{\mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_m\}$.

1: **for** $k = 1$ to $|\mathbf{P}_0|$ **do**

2: $\mathbf{P}_k \leftarrow \mathbf{P}_0[k]$;

3: **end for**

4: Calculate the inter-cluster distance d_{inter} and intra-cluster distance d_{intra} for $\mathbf{P}$;

$\mathbf{P} = \{\mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_n\}$

$d_{inter} = \sum_{\mathbf{P}_k, \mathbf{P}_s \in \mathbf{P}} D(\mathbf{P}_k, \mathbf{P}_s)$

$d_{intra} = \sum_{\mathbf{P}_k \in \mathbf{P}} \sum_{\vec{t}_i, \vec{t}_r \in \mathbf{P}_k} d(\vec{t}_i, \vec{t}_r)$

5: **while** $d_{inter} > d_{intra}$ **do**

6: Merge clusters $\mathbf{P}_k$ and $\mathbf{P}_s$ where $D(\mathbf{P}_k, \mathbf{P}_s) = \min_{k \neq s} D(\mathbf{P}_k, \mathbf{P}_s)$;

7: Update d_{inter} and d_{intra} ;

8: **end while**

As is later shown in Sect. 5.4.3, the improved method could handle high dimensions well, which makes it a compatible tool to fit in NNE's optimization.

3.3 Subpopulation movement and archive scheme

DEs, while powerful, often suffer from premature convergence and search stagnation. To address this challenge, drawing lessons from the methods in ANDE [22], an aux-

iliary movement and archiving technique is proposed, as depicted in Algorithm 3. This technique categorizes subpopulations into three states: stagnating, converging, and converged.

Algorithm 3 Auxiliary movement and archive

Input: $\mathbf{P}_k^q$ (the k th subpopulation in its q th generation), $\vec{t}_k^q$ and $\vec{t}_k^{q-1}$ (the best solutions of the k th subpopulation in its q th and $(q - 1)$ th generation), $f(\cdot)$ (the fitness function), ϵ_0 and θ (small given numbers), St (a given integer), η (the count of generations that the best individual of $\mathbf{P}_k$ stops updating).

Output: The new subpopulation $\mathbf{P}_k^q$ and the updated archive of promising solutions.

```

1: Initialize Status as None;
2: Calculate  $R(\mathbf{P}_k^q)$  by Eq. (19);
3: if  $f(\vec{t}_{best}^{\mathbf{P}_k^q}) - f(\vec{t}_{best}^{\mathbf{P}_k^{q-1}}) \leq \epsilon_0$  then
4:    $\eta++$ 
5:   if  $R(\mathbf{P}_k^q) \leq \theta$  then
6:     Status = Converged;
7:   else if  $\eta == St$  then
8:     Status = Stagnating;
9:   end if
10: else
11:   Status = Converging;
12: end if
13: if Status  $\neq$  None then
14:    $\eta = 0$ ;
15: end if
16: if Status = Converging then
17:   Apply Brownian movement by Eq. (18);
18: else if Status = Stagnating then
19:   Apply uniform random movement by Eq. (16);
20: else if Status = Converged then
21:   Add  $\vec{t}_k^q$  to the external archive;
22:   Reinitialize  $\mathbf{P}_k^q$ ;
23: end if

```

For a subpopulation $\mathbf{P}_k$ in its q th generation, its best individual is denoted as $\vec{t}_{best}^{\mathbf{P}_k^q}$. Subpopulation $\mathbf{P}_k^q$ is identified as stagnating if the fitness value of its best individual stops updating for St consecutive generations. The parameter St determines the algorithm's tolerance for temporary stagnation, influencing the convergence speed. In this article, St is set to six to balance convergence and stagnation tolerance. Specifically, if for six successive six generations, $f(\vec{t}_{best}^{\mathbf{P}_k^q}) - f(\vec{t}_{best}^{\mathbf{P}_k^{q-1}}) \leq \epsilon_0$, subpopulation $\mathbf{P}_k^q$ is classified as stagnating, with $\epsilon_0 = 0.001$. To alleviate stagnation, a uniform random movement

$$\vec{t}_{best}^{\mathbf{P}_k^q} = U[\vec{t}_{min}, \vec{t}_{max}] \quad (16)$$

is applied to $\mathbf{P}_k^q$, where $\vec{t}_{min}$ and $\vec{t}_{max}$ are the pre-given lower and upper bounds for each individual and $U[\vec{t}_{min}, \vec{t}_{max}]$ represents an equiprobability distribution from $\vec{t}_{min}$ to $\vec{t}_{max}$. If some dimensions of $\vec{t}_{best}^{\mathbf{P}_k^q}$ are outside $[\vec{t}_{min}, \vec{t}_{max}]$, they will

be reset to the feasible boundary. After the motion, a greedy choice is made between $\vec{t}_k^q$ and $\vec{t}_{best}^{\mathbf{P}_k^q}$ as in

$$\vec{t}_{best}^{\mathbf{P}_k^q} = \begin{cases} \vec{t}_{best}^{\mathbf{P}_k^q}, & \text{if } f(\vec{t}_{best}^{\mathbf{P}_k^q}) > f(\vec{t}_{best}^{\mathbf{P}_k^{q-1}}) \\ \vec{t}_{best}^{\mathbf{P}_k^{q-1}}, & \text{otherwise} \end{cases} \quad (17)$$

Following this movement, the stagnating subpopulation is capable of dropping to some more promising region and transforming into a converging subpopulation.

In contrast, a subpopulation $\mathbf{P}_k^q$ is considered converging if its best solution surpasses the previous best solution, i.e., if $f(\vec{t}_{best}^{\mathbf{P}_k^q}) - f(\vec{t}_{best}^{\mathbf{P}_k^{q-1}}) > \epsilon_0$. To prevent premature convergence and enhance diversity, the best individual $\vec{t}_k^q$ undergoes a Brownian movement

$$\vec{t}_{best}^{\mathbf{P}_k^q} = N(\vec{t}_{best}^{\mathbf{P}_k^q}, R(\mathbf{P}_k^q)), \quad (18)$$

where $R(\mathbf{P}_k^q)$ is the radius of $\mathbf{P}_k^q$, calculated by

$$R(\mathbf{P}_k^q) = \frac{1}{|\mathbf{P}_k^q| - 1} \sqrt{\sum_{i=1}^{|\mathbf{P}_k^q|} (\vec{t}_{best}^{\mathbf{P}_k^q} - \vec{t}_i^{\mathbf{P}_k^q})^2}. \quad (19)$$

Subsequent steps, including outlier reset and greedy selection, mirror those for stagnating subpopulations.

A subpopulation is recognized as a converged one when its best solution stops updating and its radius falls below a threshold θ , which is set to 0.01. To preserve the optimal solution from a converged subpopulation, it is stored to an external archive for subsequent selection. Simultaneously, the subpopulation undergoes an entire reinitialization process, maintaining its original size, with all individuals randomly regenerated within the range of $[\vec{t}_{min}, \vec{t}_{max}]$. In scenarios where a subpopulation has converged, signifying that all individuals have congregated around a peak solution, continued exploitation of this localized region can be computationally inefficient. To mitigate redundant searches within such a subpopulation and to enhance the overall diversity of the population, thereby bolstering the algorithm's capacity to explore uncharted territories within the problem space, a complete reinitialization of the converged subpopulation is deemed necessary.

After the evolutionary process concludes and the archive is populated with potential solutions, a selection strategy outlined in Algorithm 4 is employed to identify the most promising solutions from both the last population and the archive. Firstly, the best individuals from each subpopulation of the last generation are extracted and merged into the archive. The solution exhibiting the highest fitness value within the entire archive is denoted as $\vec{t}_{best}$. Then, within a controlled range of solutions (between 3 and 15), those with

lower fitness values (i.e., $\vec{t}_i$ s where $f(\vec{t}_{best}) - f(\vec{t}_i) > \epsilon_1$) are excluded to enhance the final accuracy. Following this, the distances between each pair of solutions are computed. If the distance between two solutions $\vec{t}_i$ and $\vec{t}_j$ is below a threshold (i.e., $d_{ij} < \epsilon_2 \cdot D$, where D represents the dimension of the solution space), the solution with the lower fitness value is eliminated. This step aims to preserve solution diversity and prevent the inclusion of similar base networks in subsequent processes. In this article, the values of ϵ_1 and ϵ_2 are set to 0.05 and 0.01, respectively.

Algorithm 4 Archive selection

Input: $\mathbf{P}_1$ (set of solutions in the archive), $\mathbf{P}_2$ (set of solutions in the last population), $f(\cdot)$ (the fitness function), ϵ_1 and ϵ_2 (small predefined values), D (dimension of the solution space).

Output: Selected candidate NN sets $\mathbf{P}_c$.

```

1: Initialize  $i = 0$ ;
2: Merge  $\mathbf{P}_c \leftarrow \mathbf{P}_1 \cup \mathbf{P}_2$ ;
3: Sort  $\mathbf{P}_c$  in ascending order and set  $\vec{t}_{best}$  as the best individual in  $\mathbf{P}_c$ ;
4: for  $i = 1$  to  $|\mathbf{P}_c|$  do
5:   if  $|\mathbf{P}_c| \leq 3$  then
6:     break;
7:   end if
8:   if  $f(\vec{t}_{best}) - f(\vec{t}_i) > \epsilon_1$  then
9:     Remove  $\vec{t}_i$  from  $\mathbf{P}_c$ ;
10:  end if
11: end for
12: Sort  $\mathbf{P}_c$  in descending order;
13: while  $|\mathbf{P}_c| > 3$  do
14:   for  $i = 1$  to  $|\mathbf{P}_c|$  do
15:     for  $j = i + 1$  to  $|\mathbf{P}_c|$  do
16:       Calculate the Euclidean distance  $d_{ij}$ ;
17:       if  $d_{ij} < \epsilon_2 \cdot D$  then
18:         Remove  $\vec{t}_j$  from  $\mathbf{P}_c$ ;
19:       end if
20:     end for
21:   end for
22: end while
23: if  $|\mathbf{P}_c| > 15$  then
24:   for  $i = 16$  to  $|\mathbf{P}_c|$  do
25:     Remove  $\vec{t}_i$  from  $\mathbf{P}_c$ ;
26:   end for
27: end if

```

3.4 Population augmentation

To address population stagnation, a new subpopulation is introduced to the stagnant population, as outlined in Algorithm 5. Initially, the algorithm evaluates whether there has been an improvement in the fitness value of the best solution within the current generation of the population, denoted as $f(\vec{t}_{best}^{\mathbf{P}^q})$, compared to the previous generation, denoted as $f(\vec{t}_{best}^{\mathbf{P}^{q-1}})$, i.e., if $f(\vec{t}_{best}^{\mathbf{P}^q}) - f(\vec{t}_{best}^{\mathbf{P}^{q-1}}) > \epsilon_0$. If not, the algorithm tracks the discovery of optima across all subpopulations by monitoring a variable G_q . The counter η is incremented if G_q matches the value of G_{q-1} . A popula-

tion is deemed stagnant if the value of G_q remains constant for St consecutive generations. To rejuvenate the population and enhance its diversity, an additional subpopulation $\mathbf{P}'$ is generated and merged with the existing population. The size of $\mathbf{P}'$ matches the average subpopulation size of the current generation. In the augmentation strategy, the parameter settings for the thresholds are defined as $\epsilon_0 = 0.001$ and $St = 6$, aligning with the parameters utilized for subpopulation movement as detailed in Sect. 3.3. This augmentation process aims to introduce fresh genetic individuals into the population, thereby mitigating stagnation and fostering evolutionary progress.

Algorithm 5 Population augmentation

Input: $\mathbf{P}^q$ (the population in its q th generation), $f(\cdot)$ (the fitness function), $\vec{t}_{best}^{\mathbf{P}^q}$ and $\vec{t}_{best}^{\mathbf{P}^{q-1}}$ (the best solutions of the population in its q th and $(q - 1)$ th generation), ϵ_0 (a small given number), η (the count of generations that the best individual of $\mathbf{P}^q$ stops updating), St (a given integer).

Output: The augmented population $\mathbf{P}^{q+1}$.

```

1: Initialize  $G_t = 0$ ;
2: if  $f(\vec{t}_{best}^{\mathbf{P}^q}) - f(\vec{t}_{best}^{\mathbf{P}^{q-1}}) \leq \epsilon_0$  then
3:   for  $j = 1$  to  $|\mathbf{P}^q|$  do
4:     if  $f(\vec{t}_{best}^{\mathbf{P}^q}) - f(\vec{t}_{best}^{\mathbf{P}^j}) \leq \epsilon_0$  then
5:        $G_t++$ ;
6:     end if
7:   end for
8:   if  $G_q == G_{q-1}$  then
9:      $\eta++$ ;
10:  else
11:     $\eta = 0$ ;
12:  end if
13: end if
14: if  $\eta == St$  then
15:   create a random subpopulation  $\mathbf{P}'$ ;
16:    $\mathbf{P}^{q+1} = \mathbf{P}^q \cup \mathbf{P}'$ ;
17:    $\eta = 0$ ;
18: end if

```

4 The ensemble learning algorithm based on enhanced MMDE and ELM

This section provides an overview of the motivation behind and the workflow of the proposed MMDE-ELM-NNE algorithm. It delves into the transformation of an MMOP into an NNE optimization problem, addressing key aspects such as the fitness function and the encode and decode strategies.

4.1 Motivation

(1) Since the NNE problem can be regarded as an MMOP, an enhanced MMDE is proposed to fully fit into classification problems on high-dimensional datasets.

(2) The enhanced MMDE algorithm enables the simultaneous optimization of both the structural and input parameters of an SLFN. Subsequently, its output parameters are computed by the MP pseudo-inverse method employed in the ELM. In this way, multiple base NNs with varying numbers of hidden neurons can be obtained in one run for a given classification problem, so that the diversity of base NNs can be kept and generalization ability of the NNE are ensured to be strong.

(3) Integration of the ELM algorithm into the optimization process of MMDE enhances computational efficiency and classification accuracy due to its non-iterative learning nature.

4.2 Encode and decode strategies

Determining the optimal structure for an NN has long been a challenging task. A network that is too simplistic may lack the capacity to effectively model the problem at hand, while an overly complex network may lead to overfitting and subpar generalization performance. Selecting the appropriate network topology is crucial, as the network's ability to process information is inherently linked to its structure. Two primary strategies have been devised to adjust a network's structure: pruning algorithms [51, 59] and constructive algorithms [31, 60]. However, NN structures generated through these methods often converge to local optima. In this work, an innovative encoding strategy is proposed to realize the concurrent optimization of both the topology and weight parameters of a base NN. Additionally, an ELM architecture is incorporated to enhance the computational capacity of the network.

4.2.1 Encode strategy

In the proposed MMDE-ELM-NNE framework, the topology and weight information of each SLFN are encoded within the same individual, allowing for simultaneous optimization. Each individual is represented as a real-number vector, comprising the state parameters of hidden layer neurons, the weights between the input and hidden layers, and the hidden layer biases.

For an SLFN with n neurons in the input layer, L neurons in the hidden layer, and m neurons in the output layer, an individual is represented as $\vec{t} = [\vec{\delta}^T, \vec{w}_1^T, \vec{w}_2^T, \dots, \vec{w}_L^T, \vec{b}^T]$, where $\vec{\delta} \in \mathbf{R}^L$, $\vec{w}_1, \vec{w}_2, \dots, \vec{w}_L \in \mathbf{R}^n$, and $\vec{b} \in \mathbf{R}^L$. The dimensionality of this encoded vector can be formulated as $D = (n + 2) \times L$. The elements of each vector are floating-point numbers within the range of $[-1, 1]$.

Figure 1 illustrates an example of an SLFN with an input layer comprising two neurons, a hidden layer with five neurons, and an output layer with two neurons. The network's parameter configuration is tabulated in Table 1, detailing key

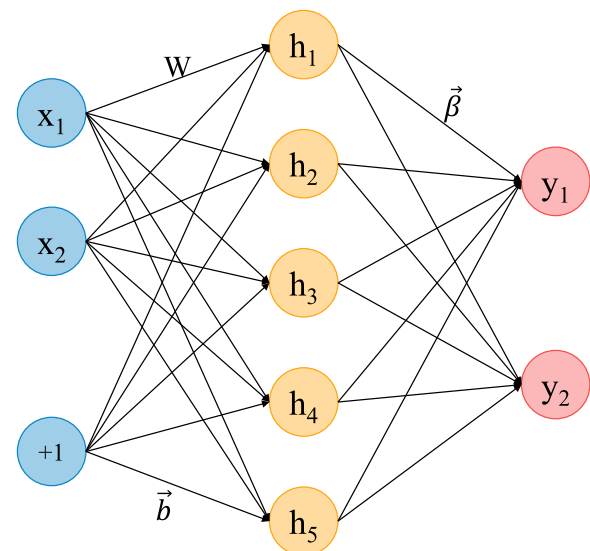


Fig. 1 An SLFN model with three layers

Table 1 The parameters to be optimized in the SLFN model illustrated in Fig. 1

$\vec{\delta}^T$	δ_1	δ_2	δ_3	δ_4	δ_5
$\mathbf{W}$	w_{11}	w_{21}	w_{31}	w_{41}	w_{51}
	w_{12}	w_{22}	w_{32}	w_{42}	w_{52}
$\vec{b}^T$	b_1	b_2	b_3	b_4	b_5

elements such as the activation states of neurons in the hidden layer denoted by $\vec{\delta}$, the weight matrix linking the input and hidden layers represented as $\mathbf{W} = [\vec{w}_1, \vec{w}_2, \vec{w}_3, \vec{w}_4, \vec{w}_5]$, and the bias vector to the hidden layer, denoted as $\vec{b}$. These parameters are sequentially encoded into a twenty-dimensional vector $\vec{t} = [\vec{\delta}^T, \vec{w}_1^T, \vec{w}_2^T, \vec{w}_3^T, \vec{w}_4^T, \vec{w}_5^T, \vec{b}^T]$.

4.2.2 Decode strategy

The activation state of the i th hidden layer neurons can be obtained from the state parameters δ_i by

$$s_i = \begin{cases} 0, & \text{if } \text{sigmoid}(\delta_i) < \text{rand}(0, 1) \\ 1, & \text{otherwise} \end{cases}, \quad (20)$$

where $\text{rand}(0, 1)$ is a random number between $(0, 1)$, and

$$\text{sigmoid}(\delta_i) = \frac{1}{1 + \exp(-\delta_i)}. \quad (21)$$

In this way, for every floating-point number δ_i in the range of $[-1, 1]$, there is a corresponding binary value s_i (taking values of 0 or 1), wherein s_i dictates the activation status

of the i th neuron. Specifically, the i th neuron is considered active if $s_i = 1$ and inactive if $s_i = 0$. Notably, deactivation of the i th neuron results in the elements in the i th column of the weight matrix $\mathbf{W}$ and the i th bias of $\vec{b}$ in the network being set to zero.

Following this process, the network's structure and input parameters are determined, while the output parameters β can be calculated according to Eq. (13).

4.3 Fitness function

Tailoring the nature of problems involving maximization optimization and classification, the fitness function is defined as the negative of the cross-entropy loss, as illustrated in Eq. (22):

$$f(\vec{x}) = \frac{1}{N \cdot m} \sum_{i=1}^N \sum_{j=1}^m y_{ij} \log(o_{ij}), \quad (22)$$

where N represents the number of training samples, and m denotes the number of classes.

4.4 Workflow of the proposed MMDE-ELM-NNE method

The network structure and input parameters of an SLFN are optimized by the enhanced MMDE, and its output parameters are calculated by the MP pseudo-inverse method in ELM. These optimization processes yield multiple optimal solutions simultaneously, leading to the generation of a set of diverse SLFNs upon decoding. Subsequently, a refined subset comprising optimal SLFNs is strategically selected to serve as the base NNs for ensemble learning, culminating in the development of the MMDE-ELM-NNE algorithm, as outlined in Algorithm 6. The detailed procedures are elaborated as follows:

The structural parameters $\vec{\delta}$ and input parameters $\mathbf{W}$ and $\vec{b}$ of an SLFN are encoded into a D-dimensional vector $\vec{t}$ following the predefined encoding strategy, while the output parameters β are computed utilizing the MP pseudo-inverse method integrated within ELM, as depicted in Algorithm 1. Each vector $\vec{t}$ represents an individual within the population, with the loss function defined by Eq. (22) acting as the fitness function to be maximized. This transformation converts the initial complex NNE optimization problem into an MMOP, which can be effectively solved by applying the enhanced MMDE algorithm.

The enhanced MMDE algorithm commences by randomly initializing a population $\mathbf{P}_0$ within the specified boundary $[\vec{t}_{min}, \vec{t}_{max}] = [-1, 1]$. Following this, the population undergoes an improved heuristic clustering process based on Algorithm 2, resulting in the division of $\mathbf{P}_0$ into an adap-

Algorithm 6 MMDE-ELM-NNE

Input: a training set $\{(\vec{x}_j, \vec{y}_j) | \vec{x}_j \in \mathbf{R}^n, \vec{y}_j \in \mathbf{R}^m\}_{j=1}^N$, the maximum fitness evaluations $maxFEs$, the population size N , an activation function $g(\cdot)$, and the parameters in MMDE.

Output: The classification result.

```

1: Initialize  $FE = 0$ ;
2: Create a random initial population  $\mathbf{P}_0$ ;
3:  $\{\mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_m\} \leftarrow \mathbf{P}_0$ ;
   //cluster  $\mathbf{P}_0$  according to Algorithm 2;
4: while  $FE < maxFEs$  do
5:   for each subpopulation  $\mathbf{P}_k$  do
6:     for each individual  $\vec{t}_i$  do
7:       Update  $F$  and  $CR$  according to Eqs. (1) and (2);
8:       Generate a mutant vector using Eq. (3);
9:       Generate a trial vector by Eq. (4);
10:      Decode the trial vector and calculate the output parameters
        using the MP pseudo-inverse method employed in ELM;
11:      Perform selection based on Eq. (5);
12:    end for
13:  Perform auxiliary movement and archiving as per Algorithm 3;
14:  end for
15:  Perform augmentation following Algorithm 5;
16: end while
17: Perform solution selection according to Algorithm 4;
18: Conduct a majority vote on the outputs of the trained NNs.
```

tive number of subpopulations $\{\mathbf{P}_1, \mathbf{P}_2, \dots, \mathbf{P}_m\}$. Before the termination of the $maxFEs$ threshold is reached, the population enters a loop wherein each subpopulation sequentially undergoes mutation, crossover, and selection processes on individuals using adaptive F and CR values. Subsequently, the subpopulations identified to be stagnating, converging, or already converged are subjected to specific procedures as delineated in Algorithm 3. Stagnating subpopulations employ uniform random movement to evade local optima, while converging subpopulations utilize Brownian movement to boost diversity. Converged subpopulations transfer the best individual to an external archive before being fully reinitialized for further exploration. In case of the whole population's stagnation indicated by the lack of improvement in the best solution for St generations, a new randomly generated subpopulation is introduced to enhance diversity and counteract stagnation as described in Algorithm 5.

Upon completion of the evolutionary process, an archive selection strategy detailed in Algorithm 4 is employed to identify optimal non-overlapping solutions. The most promising solutions are selected from the optima in each subpopulation of the latest generation and the archive gathered from former converged subpopulations. Each optimal solution, along with its output parameters, corresponds to a trained NN, and each NN's output contributes to the predicted category by a majority voting mechanism. This involves aggregating the individual classification results from each NN, with the category receiving the highest number of votes

Table 2 Characteristics and parameter settings of 20 benchmark functions of CEC2013

Function	Dimension	Number of global optima	<i>NP</i>	<i>max FEs</i>
F1	1	2	100	5.00E+04
F2	1	5	100	5.00E+04
F3	1	1	100	5.00E+04
F4	2	4	100	5.00E+04
F5	2	2	100	5.00E+04
F6	2	18	100	2.00E+05
F7	2	36	300	2.00E+05
F8	3	81	300	4.00E+05
F9	3	216	300	4.00E+05
F10	2	12	100	2.00E+05
F11	2	6	200	2.00E+05
F12	2	8	200	2.00E+05
F13	2	6	200	2.00E+05
F14	3	6	200	4.00E+05
F15	3	8	200	4.00E+05
F16	5	6	200	4.00E+05
F17	5	8	200	4.00E+05
F18	10	6	200	4.00E+05
F19	10	8	200	4.00E+05
F20	20	8	200	4.00E+05

being designated as the final classification outcome, a strategy commonly known as the "winner-take-all" approach.

5 Experiments

To demonstrate the superiority and effectiveness of MMDE-ELM-NNE, a comparative experiment was conducted to assess its performance against other state-of-the-art algorithms. Furthermore, additional experiments were carried out to delve deeper into the performance of MMDE-ELM-NNE. These experiments focus on investigating the impact of various factors, including the number of base NNs in NNE, the number of hidden neurons, the population size, the maximum number of generations, and the effectiveness of the improved heuristic clustering method.

5.1 Experimental studys on MMOPs

In this segment, the performance of the proposed MMDE algorithm is meticulously appraised through the utilization of the CEC2013 benchmark suite [33]. This suite encompasses 20 multimodal test functions, which are categorically partitioned into three distinct groups. The initial ten functions are well recognized low dimensional basic functions, serving as fundamental testbeds for algorithm evaluation. The subsequent five functions are low dimensional composite functions, characterized by the presence of numerous

local optima, thereby posing significant challenges to optimization algorithms. The remaining five functions are high dimensional composite functions, adding another layer of complexity due to the curse of dimensionality. The dimensions and the number of global optima for these functions are elaborately detailed in Table 2.

The experimental results of MMDE are compared with eleven classical and state-of-the-art multimodal optimization algorithms, as follows.

- CDE [49]: Crowding-based Differential Evolution;
- SDE [32]: Speciation-based Differential Evolution;
- NCDE [44]: Crowding-based Differential Evolution with neighborhood mutation;
- NSDE [44]: Speciation-based Differential Evolution with neighborhood mutation;
- dADE/nrand/l [11]: Dynamic archive niching Differential Evolution with adaptive parameters;
- ANLDE [45]: Differential Evolution with adaptive neighborhood mutation and local search;
- PRMDE [61]: Proximity ranking-based multimodal Differential Evolution;
- LBPADDE [64]: Local binary pattern-based adaptive Differential Evolution;
- AED-DDE [53]: Adaptive estimation distribution distributed Differential Evolution;
- VNCDE [63]: Parameter-free Voronoi neighborhood Differential Evolution;

Table 3 PR and SR of MMDE on 20 benchmark functions

ϵ	F1		F2		F3		F4		F5	
	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR
1.0E-01	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
1.0E-02	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
1.0E-03	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
1.0E-04	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
1.0E-05	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
ϵ	F6		F7		F8		F9		F10	
	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR
1.0E-01	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
1.0E-02	1.000	1.000	1.000	1.000	1.000	1.000	0.712	0.000	1.000	1.000
1.0E-03	1.000	1.000	0.964	0.800	1.000	1.000	0.646	0.000	1.000	1.000
1.0E-04	1.000	1.000	0.964	0.360	1.000	1.000	0.646	0.000	1.000	1.000
1.0E-05	0.980	0.960	0.950	0.293	1.000	1.000	0.624	0.000	1.000	1.000
ϵ	F11		F12		F13		F14		F15	
	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR
1.0E-01	1.000	1.000	0.863	0.000	1.000	1.000	1.000	1.000	1.000	1.000
1.0E-02	1.000	1.000	0.863	0.000	0.816	0.000	0.716	0.000	0.735	0.000
1.0E-03	1.000	1.000	0.863	0.000	0.816	0.000	0.716	0.000	0.701	0.000
1.0E-04	1.000	1.000	0.834	0.000	0.816	0.000	0.714	0.000	0.701	0.000
1.0E-05	1.000	1.000	0.747	0.000	0.766	0.000	0.704	0.000	0.623	0.000
ϵ	F16		F17		F18		F19		F20	
	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR
1.0E-01	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.454	0.400
1.0E-02	0.667	0.000	0.667	0.000	0.670	0.000	0.731	0.000	0.301	0.000
1.0E-03	0.667	0.000	0.667	0.000	0.667	0.000	0.463	0.000	0.267	0.000
1.0E-04	0.667	0.000	0.667	0.000	0.667	0.000	0.463	0.000	0.242	0.000
1.0E-05	0.667	0.000	0.413	0.000	0.633	0.000	0.298	0.000	0.061	0.000

- ANDE [52]: Automatic niching Differential Evolution with contour prediction approach.

All algorithms terminate when the maximum number of function evaluations (MaxFEs) is exhausted. For each test function, specific values of MaxFEs and population size (NP) are provided in Table 2. Additionally, due to the stochastic nature of the algorithms, each algorithm was independently executed 30 times. Two evaluation metrics—Peak Ratio (PR) and Success Rate (SR)—were employed to compare MMDE with other niching algorithms. The Peak Ratio (PR) is defined as the average percentage of global optima identified relative to the total known global optima, given a fixed MaxFEs and a specific precision level ϵ . The Success Rate (SR) is the percentage of successful runs, where a successful run is defined as identifying all known global optima. In the experiments, five precision levels ($\epsilon = 1.0E-01, 1.0E-02, 1.0E-03, 1.0E-04$, and $1.0E-05$) were used to evaluate the ability of niching algorithms to locate multiple global

optima. Due to space constraints, results for $\epsilon = 1.0E-04$ are primarily reported unless otherwise specified. The results of MMDE in terms of PR and SR across all five precision levels for F1-F20 are presented in Table 3. As shown in the table, MMDE successfully identified all global optima for 18 functions at $\epsilon = 1.0E-01$, demonstrating its effectiveness in solving MMOPs. To further compare the performance of MMDE with other multimodal optimization algorithms, Table 4 provides a comparative analysis of PR and SR at the precision level $\epsilon = 1.0E-04$. For clarity, the best-performing PRs are highlighted in bold. As shown in Table 4, the proposed MMDE algorithm achieved the best PRs for 14 out of 20 test functions and successfully located all known optima for 9 of the 20 test functions. In summary, under the same MaxFEs, MMDE outperformed most of the compared niching algorithms across evaluation metrics such as PR and SR. This demonstrates the superiority of MMDE in multimodal optimization tasks.

Table 4 PR and SR of twelve comparing algorithms on 20 benchmark functions at accuracy level $\varepsilon = 1.0E-04$

	MMDE		CDE		SDE		NCDE		NSDE		dADE/nrand/1	
	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR
F1	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
F2	1.000	1.000	1.000	1.000	0.920	0.800	1.000	1.000	0.856	0.544	1.000	1.000
F3	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
F4	1.000	1.000	1.000	1.000	0.366	0.000	1.000	1.000	0.822	0.436	1.000	1.000
F5	1.000	1.000	1.000	1.000	0.910	0.824	1.000	1.000	0.865	0.632	1.000	1.000
F6	1.000	1.000	0.988	0.920	0.129	0.000	0.455	0.000	0.176	0.000	0.976	0.760
F7	0.964	0.360	0.754	0.000	0.122	0.000	0.882	0.000	0.238	0.000	0.827	0.000
F8	1.000	1.000	0.010	0.000	0.176	0.000	0.020	0.000	0.204	0.000	0.962	0.360
F9	0.646	0.000	0.467	0.000	0.231	0.000	0.539	0.000	0.166	0.000	0.422	0.000
F10	1.000	1.000	1.000	1.000	0.158	0.000	0.856	0.720	0.357	0.000	1.000	1.000
F11	1.000	1.000	0.322	0.000	0.334	0.000	0.788	0.045	0.482	0.000	0.667	0.000
F12	0.834	0.000	0.060	0.000	0.306	0.000	0.179	0.000	0.364	0.000	0.700	0.000
F13	0.816	0.000	0.134	0.000	0.312	0.000	0.667	0.000	0.276	0.000	0.667	0.000
F14	0.714	0.000	0.018	0.000	0.233	0.000	0.667	0.000	0.258	0.000	0.667	0.000
F15	0.701	0.000	0.007	0.000	0.025	0.000	0.325	0.000	0.189	0.000	0.619	0.000
F16	0.667	0.000	0.000	0.000	0.124	0.000	0.667	0.000	0.184	0.000	0.667	0.000
F17	0.667	0.000	0.000	0.000	0.128	0.000	0.187	0.000	0.176	0.000	0.370	0.000
F18	0.667	0.000	0.188	0.000	0.057	0.000	0.503	0.000	0.377	0.000	0.623	0.000
F19	0.463	0.000	0.000	0.000	0.209	0.000	0.346	0.000	0.089	0.000	0.029	0.000
F20	0.242	0.000	0.000	0.000	0.000	0.000	0.220	0.000	0.139	0.000	0.002	0.000
Mean	0.819	0.468	0.447	0.346	0.337	0.181	0.615	0.288	0.411	0.181	0.710	0.356

	ANLDE		PRMDE		LBPADE		AED-DDE		VNCDE		ANDE	
	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR	PR	SR
F1	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
F2	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
F3	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
F4	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
F5	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
F6	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.967	0.540	1.000	1.000
F7	0.886	0.000	0.988	0.686	0.889	0.000	0.838	0.039	0.746	0.000	0.933	0.176
F8	0.999	0.902	1.000	0.980	0.575	0.000	0.747	0.000	0.558	0.000	0.944	0.078
F9	0.454	0.000	0.706	0.000	0.476	0.000	0.384	0.000	0.298	0.000	0.512	0.000
F10	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
F11	1.000	1.000	0.997	0.980	0.674	0.000	1.000	1.000	1.000	1.000	1.000	1.000
F12	0.993	0.961	1.000	1.000	0.750	0.000	1.000	1.000	0.993	0.940	0.868	0.940
F13	1.000	1.000	0.667	0.000	0.667	0.000	0.686	0.000	0.687	0.000	0.687	0.000
F14	0.667	0.000	0.667	0.000	0.667	0.000	0.667	0.000	0.667	0.000	0.667	0.000
F15	0.667	0.000	0.615	0.000	0.654	0.000	0.637	0.000	0.748	0.000	0.632	0.000
F16	0.618	0.000	0.667	0.000	0.667	0.000	0.667	0.000	0.667	0.000	0.667	0.000
F17	0.667	0.000	0.507	0.000	0.532	0.000	0.375	0.000	0.593	0.000	0.397	0.000
F18	0.660	0.000	0.667	0.000	0.667	0.000	0.654	0.000	0.667	0.000	0.654	0.000
F19	0.463	0.000	0.453	0.000	0.475	0.000	0.375	0.000	0.475	0.000	0.363	0.000
F20	0.255	0.000	0.252	0.000	0.275	0.000	0.250	0.000	0.290	0.000	0.248	0.000
Mean	0.816	0.543	0.809	0.532	0.748	0.350	0.764	0.452	0.768	0.424	0.779	0.460

Table 5 Characteristics of benchmark classification problems

Datasets	Samples	Features	Classes
Heart	270	13	2
Pima	768	8	2
Wine	178	13	3
Balance	625	4	3
Iris	150	4	3
Seeds	210	7	3
Breast	699	9	2
Haberman	306	3	2
Blood	748	4	2
Australian	690	14	2
Liver	345	6	2
Hepatitis	155	19	2
Diabetic	1151	19	2

5.2 Experimental setup for classification

A total of 13 benchmark classification datasets are picked from the UCI machine learning repository [4] for experiments. These datasets span diverse fields. Table 5 provides an overview about the datasets, presents an overview of the datasets, detailing various sample sizes, input attribute sizes, and output class sizes. For the data preprocessing, all features are normalized to the range of [0, 1] using the min-max normalization method.

For accurate estimation of generalization performance, ten-fold-cross-validation is employed in the experiments. This method randomly divides the dataset into ten equal subsets, with nine utilized for training and one for testing each time. The average of all experimental results is calculated across 30 independent runs.

Table 6 showcases the key parameter values used in the proposed algorithm and the comparative ones, including the population size NP , maximum number of fitness evaluations $maxFEs$, maximum number of hidden nodes, and the number of NNs participating in the final vote. The NNs to be ensembled in the proposed algorithm are all SLFNs. The numbers of nodes in the input and output layers, denoted by n and m respectively, are determined by the features and classes present in the dataset. While setting the number of hidden neurons manually is not mandatory due to the adaptability of the MMDE in determining its optimal value, it is essential to consider the significant increase in dimensionality and the subsequent rise in computational costs as L grows. Therefore, in this study, we specify the maximum number of hidden neurons $L = 5$ for all participating algorithms to ensure fair competition. Additionally, to maintain fairness across all algorithms in the comparison, a maximum of 20 base NNs is provided for each algorithm.

Table 6 Key control parameter settings of all the compared algorithms in Table 7

Control parameter	Value
Population size NP	100
Maximum fitness evaluations $maxFEs$	13000
Maximum base NNs for NNE	20
Maximum hidden nodes L	5

The implementation of the proposed algorithm is carried out using Python 3.9 on hardware featuring an AMD Ryzen 5 4600 G with Radeon Graphics operating at 3.70 GHz.

5.3 Comparison with other algorithms on classification problems

The MMDE-ELM-NNE algorithm has been rigorously compared with thirteen other algorithms, including variations of the MMDE and ELM frameworks such as MMDE-NN, which combines the enhanced MMDE method with a traditional SLFN, and MMDE-NNE, which creates multiple SLFNs in a single run and consolidates them into an ensemble learning framework. Additionally, ELM-NN, which applies the ELM algorithm to train a single neural network, and ELM-NNE, an ensemble of independently trained ELM-NN models, are also considered. The comparison also encompasses well-regarded metaheuristic algorithms like the Genetic Algorithm (GA) [16, 18], Particle Swarm Optimization (PSO) [29], and Artificial Bee Colony (ABC) [27, 28], as well as cutting-edge metaheuristic training algorithms from the literature, including PSO variants such as LFPSO [17], PSOLF [24], and LPSONs [48], along with the Dragonfly algorithm (DA) [36], Sine Cosine Algorithm (SCA) [37], and Grey Wolf Optimizer (GWO) [12].

Tables 7, 8 collectively detail the performance metrics of the MMDE-ELM-NNE algorithm and its counterparts. Table 7 examines the best, average, and standard deviation of classification accuracy for MMDE-ELM-NNE and seven other algorithms, including MMDE-NN, MMDE-NNE, ELM-NN, ELM-NNE, LFPSO, PSOLF, and LPSONs, across thirteen datasets, while Table 8 focuses on the same metrics for MMDE-ELM-NNE and six well-regarded metaheuristic algorithms, including GA, PSO and ABC, DA, SCA and GWO, across nine datasets. In both tables, the superior results are highlighted in bold.

To ascertain statistical significance, the performance of these algorithms on all datasets underwent rigorous evaluation using the Friedman test [13]. This test, based on ranking, compares the performance of multiple algorithms across various datasets to identify significant differences. The average ranking results of classification accuracy, including Best and Mean values, for the fourteen algorithms are detailed in

Table 7 Comparing the performance of eight algorithms: MMDE-ELM-NNE, MMDE-NN, MMDE-NNE, ELM-NN, ELM-NNE, LFPSO, PSOLF, and LPSONS on thirteen benchmark classification problems in terms of testing accuracy (%)

Dataset		MMDE-ELM-NNE	MMDE-NN	MMDE-NNE	ELM-NN	ELM-NNE	PSOLF	LFPSO	LPSONS
Heart	Best	100.00	92.59	92.59	88.89	92.59	83.14	87.64	85.39
	Mean	83.46	81.98	70.74	74.07	82.72	80.24	82.58	81.12
	Std	8.06	8.79	14.27	9.73	6.18	1.83	3.48	2.64
Pima	Best	84.21	68.42	73.68	85.53	81.58	78.23	76.95	78.26
	Mean	77.11	65.13	65.26	72.54	72.68	73.69	67.33	74.26
	Std	4.23	2.89	7.67	5.83	4.23	4.12	7.51	3.90
Wine	Best	100.00	82.35	88.24	100.00	100.00	98.11	98.11	100.00
	Mean	98.04	57.25	55.88	83.73	97.25	93.77	94.13	95.09
	Std	2.82	19.04	18.59	10.09	4.01	4.27	3.46	3.28
Balance	Best	96.77	93.55	90.32	90.32	95.16	90.90	90.37	91.44
	Mean	89.89	85.27	78.66	80.86	87.96	88.83	87.16	88.83
	Std	2.57	4.06	8.58	8.99	3.71	1.73	2.32	2.00
Iris	Best	100.00	80.00	93.33	100.00	100.00	97.77	100.00	100.00
	Mean	96.89	60.00	60.00	85.78	90.00	91.55	97.11	97.55
	Std	5.63	11.07	14.75	10.61	8.35	7.01	2.98	2.86
Seeds	Best	100.00	95.24	95.24	100.00	100.00	100.00	98.33	100.00
	Mean	96.51	71.27	67.62	91.43	92.86	94.16	92.50	92.50
	Std	3.52	14.87	18.90	7.12	5.83	3.45	3.26	4.79
Breast	Best	100.00	98.55	98.55	98.55	100.00	97.56	97.07	97.56
	Mean	96.71	95.41	82.32	93.96	96.14	95.90	95.90	96.48
	Std	2.15	2.85	12.57	3.30	1.76	1.16	0.76	0.72
Haberman	Best	90.00	86.67	83.33	86.67	83.33	78.26	78.26	80.43
	Mean	74.78	73.33	73.00	72.56	73.89	73.47	73.60	75.00
	Std	7.02	9.71	6.15	7.04	6.50	3.70	4.55	2.51
Blood	Best	86.49	82.43	85.14	85.14	83.78	79.91	82.14	81.25
	Mean	78.02	76.08	76.49	77.07	76.80	77.13	77.46	77.99
	Std	4.18	3.69	5.22	6.70	4.14	2.48	3.39	2.44
Australian	Best	94.20	89.86	89.86	88.41	91.30	89.85	88.88	89.85
	Mean	85.80	82.37	74.88	77.63	85.36	86.04	86.23	86.35
	Std	4.36	5.22	13.24	7.90	3.72	2.63	1.84	2.23
Liver	Best	79.41	73.53	70.59	73.53	73.53	73.23	70.58	73.53
	Mean	69.71	58.53	58.24	60.00	62.16	64.69	65.39	66.38
	Std	5.36	10.05	7.78	8.30	7.56	4.48	6.47	4.50
Hepatitis	Best	100.00	100.00	100.00	93.33	100.00	91.17	85.29	91.18
	Mean	86.44	80.00	79.33	80.00	79.33	79.13	79.11	79.41
	Std	9.51	10.93	13.03	5.25	10.55	8.01	4.26	3.99
Diabetic	Best	75.65	66.09	66.96	66.96	69.57	70.43	67.95	70.92
	Mean	69.74	61.65	58.41	57.65	60.90	63.92	60.78	63.83
	Std	3.22	3.21	4.92	5.29	4.84	2.33	4.02	2.21

Tables 9, 10. A lower ranking value signifies superior performance compared to other algorithms.

From both Tables 7, 8, it is evident that MMDE-ELM-NNE consistently achieves the highest testing accuracy across multiple datasets compared to the other algorithms. Tables 9, 10 indicates that MMDE-ELM-NNE ranks first in

terms of both best and mean accuracy. Meanwhile, Table 7 highlights the superiority of MMDE-ELM-NNE on 10 out of 13 datasets based on mean accuracy. Regarding the best accuracy, MMDE-ELM-NNE outperforms all other algorithms on every dataset, achieving a maximum classification accuracy of 100% on 6 out of 13 datasets. In terms of runtime, the inclu-

Table 8 Comparing the performance of seven algorithms: MMDE-ELM-NNE, GA, PSO, ABC, DA, SCA, GWO on nine benchmark classification problems in terms of testing accuracy (%)

Dataset		MMDE-ELM-NNE	GA	PSO	ABC	DA	SCA	GWO
Heart	Best	100.00	85.87	85.87	86.96	90.22	86.96	88.04
	Mean	83.46	83.19	80.40	83.19	82.43	81.88	83.99
	Std	8.06	2.40	2.65	2.61	3.31	2.54	2.19
Pima	Best	84.21	77.10	79.39	77.48	79.01	78.63	78.24
	Mean	77.11	74.11	73.12	73.16	75.22	73.66	76.58
	Std	4.23	2.16	2.45	2.76	1.81	2.22	0.79
Wine	Best	100.00	95.46	84.85	89.39	92.42	90.91	100.00
	Mean	98.04	89.39	75.96	75.35	83.38	81.67	95.00
	Std	2.82	4.54	5.23	7.00	5.58	4.81	2.82
Iris	Best	100.00	98.04	92.16	98.04	98.04	92.16	98.04
	Mean	96.89	86.93	70.98	90.07	88.30	77.97	85.49
	Std	5.63	7.32	10.32	5.98	7.90	9.87	11.18
Seeds	Best	100.00	93.06	90.28	93.06	93.06	91.67	91.67
	Mean	96.51	90.37	78.80	82.08	88.47	82.78	90.42
	Std	3.52	1.86	7.57	7.09	2.63	4.62	1.17
Breast	Best	100.00	97.90	97.90	97.90	98.32	98.32	98.32
	Mean	96.71	97.10	96.75	96.48	97.09	96.74	97.13
	Std	2.15	0.56	0.72	0.68	0.59	0.86	0.43
Haberman	Best	90.00	75.24	74.29	78.1	75.24	75.24	74.29
	Mean	74.78	72.76	72.38	73.68	73.05	72.76	72.83
	Std	7.02	0.96	0.94	2.08	0.97	1.02	0.70
Australian	Best	94.20	88.16	85.53	85.96	87.28	87.72	84.65
	Mean	85.80	83.77	82.50	82.35	83.99	83.52	83.07
	Std	4.36	1.73	1.95	2.20	1.172	2.04	1.04
Liver	Best	79.41	76.27	74.58	75.42	77.97	74.58	77.12
	Mean	69.71	67.71	65.79	63.33	70.20	66.44	72.94
	Std	5.36	4.55	4.49	5.37	3.84	4.84	2.75

Table 9 Average rankings of the eight algorithms: MMDE-ELM-NNE, MMDE-NN, MMDE-NNE, ELM-NN, ELM-NNE, LFPSO, PSOLF, and LPSONs for best and mean accuracy

Algorithm	Best	Mean
MMDE-ELM-NNE	1.65	1.46
MMDE-NN	5.12	6.08
MMDE-NNE	5.35	7.46
ELM-NN	4.15	6.19
ELM-NNE	2.96	4.04
PSOLF	5.85	4.00
LFPSO	6.31	4.23
LPSONs	4.62	2.54

sion of ELM in the MMDE framework significantly reduces the required time. The average runtime of MMDE-ELM-NNE is 41.37% of that of MMDE-NNE across all datasets. This substantial reduction is attributed to the non-iterative nature of ELM, which allows for faster training of the base neural networks.

Table 10 Average rankings of the seven algorithms: MMDE-ELM-NNE, GA, PSO, ABC, DA, SCA, GWO and mean accuracy

Algorithm	Best	Mean
MMDE-ELM-NNE	1.06	1.89
GA	4.33	3.56
PSO	6.00	6.33
ABC	4.56	5.28
DA	3.17	3.22
SCA	4.67	5.17
GWO	4.22	2.56

The Friedman tests, as reported in Tables 9, 10, demonstrate significant differences in performance across algorithms. For mean accuracy in Table 9, the test statistic is 60.43 with a p-value of 1.24×10^{-10} ; for best accuracy, the test statistic is 39.61 with a p-value of 1.49×10^{-6} . In Table 10, the mean accuracy test statistic is 30.56 with a p-value of 3.07×10^{-5} , and for best accuracy, the test statistic is 29.46

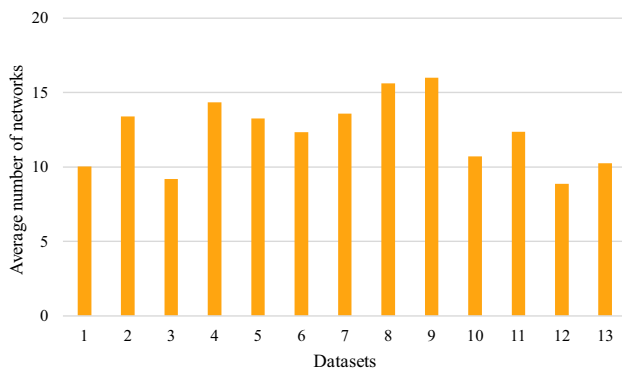


Fig. 2 Average number of NNs in NNE for each dataset obtained by MMDE-ELM-NNE (datasets: 1- Heart, 2- Pima, 3- Wine, 4- Balance, 5- Iris, 6- Seeds, 7- Breast, 8- Haberman, 9- Blood, 10- Australian, 11- Liver, 12- Hepatitis, 13- Diabetic)

with a p-value of 4.97×10^{-5} . At a significance level of 0.05, the null hypothesis of equal mean ranks among algorithms is rejected, signifying statistically significant performance disparities.

Furthermore, it has been observed that MMDE-NN outperforms MMDE-NNE, while ELM-NN underperforms compared to ELM-NNE in terms of testing accuracy. This observation suggests that the simple aggregation of NNs without considering their diversity and the specific task context may not always be advantageous, potentially hindering the generative capabilities of their NNE methods.

In contrast, MMDE-ELM-NNE, with its enhanced heuristic clustering method that effectively preserves diversity and its archive selection strategy for adaptive selection of the optimal number of NNs for ensembling, shows considerable promise.

5.4 Further analysis of MMDE-ELM-NNE

In this subsection, a comprehensive analysis of MMDE-ELM-NNE is

conducted through additional experiments designed to explore factors influencing the model's performance, including the number of base NNs, hidden neurons, population size, maximum generations, and the efficacy of the enhanced clustering method.

5.4.1 Investigation of the number of base NNs and hidden neurons

Figure 2 demonstrates the average number of base NNs in NNE obtained by MMDE-ELM-NNE across thirteen datasets. The variability in the number of base NNs in NNE, as achieved by MMDE-ELM-NNE, across different datasets is evident due to the archive selection scheme. This scheme adjusts the number of optimal solutions based on the dataset

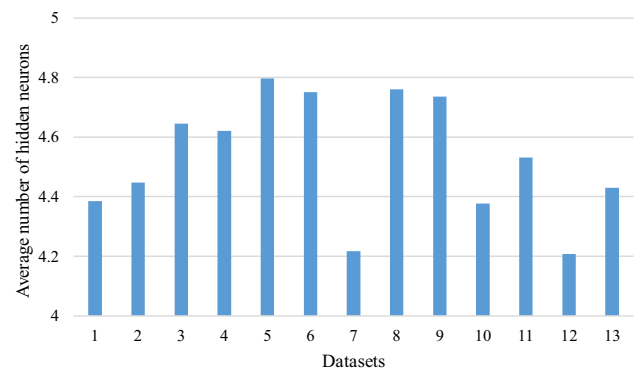


Fig. 3 Average number of hidden neurons for each dataset obtained by MMDE-ELM-NNE (datasets: 1- Heart, 2- Pima, 3- Wine, 4- Balance, 5- Iris, 6- Seeds, 7- Breast cancer, 8- Haberman, 9- Blood, 10- Australian, 11- Liver, 12- Hepatitis, 13- Diabetic)

characteristics, a critical aspect as an inappropriate number of NNs may compromise the overall ensemble effectiveness. An excessive number of NNs can introduce redundancy, while an insufficient number may limit the generalization ability of the ensemble. For instance, when comparing datasets like Wine or Hepatitis, MMDE-ELM-NNE produces much fewer NNs compared to MMDE-NNE and ELM-NNE, which use a fixed NN size of 20, yet it outperforms these rival NNEs on every dataset. This performance can be attributed to the robust multimodal optimization ability of MMDE-ELM-NNE and the superior structure of the ELM.

Figure 3 shows the average number of hidden neurons utilized in MMDE-ELM-NNE across all datasets. Each dataset adaptively employs a different number of hidden neurons, showcasing the ability of the model to customize the NN structure based on problem complexity. All networks have fewer than 5 hidden neurons for all datasets, a lower count compared to all other algorithms. This demonstrates the superiority of MMDE-ELM-NNE, as it achieves higher performance with a less complex network structure.

5.4.2 Effect of population size and maximum number of generations

Population size NP serves as a critical hyperparameter in EAs, significantly influencing their convergence and efficiency. To investigate the impact of population size on mean accuracy of MMDE-ELM-NNE, an experiment was conducted with 30 iterations of ten-fold cross-validation on six benchmark datasets, maintaining consistency with the previous experimental setup. The experiment aims to evaluate the influence of population size on MMDE-ELM-NNE by testing various population sizes (25, 50, 75, 100, 150, 200, 300). Figure 4 illustrates the changes in average classification accuracy across the six benchmark datasets as population size varies. The results indicate that the performance of

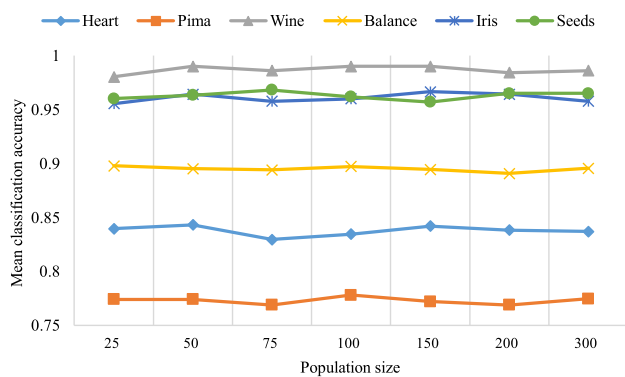


Fig. 4 Average classification accuracy across six datasets obtained by MMDE-ELM-NNE with different population sizes

MMDE-ELM-NNE remains relatively stable across different population sizes. The mean classification accuracy on each dataset shows no significant fluctuations with varying population sizes. Therefore, for fair comparison, a population size of 100 was deemed suitable for all experiments.

5.4.3 Effect of improved heuristic clustering method

Table 11 compares the effects of the original clustering method [22] and the improved clustering method across various dimensions. For the sake of generality, we consider target dimensions $n = 2, 10, 20, 200, 1000$, with an initial population size $NP = 100$. The target's lower and upper boundaries are set as $\vec{t}_{min} = [0, 0, \dots, 0]$ and $\vec{t}_{max} = [1, 1, \dots, 1]$. The random initialization and clustering process are repeated for 20 independent runs. After averaging the results of these trials, we evaluate three protocols: the number of subpopulations, the size of the largest subpopulation, and the ratio of the largest subpopulation size to the whole population size. It should be noted that the proportion of the largest subpopulation is not simply (largest subpopulation size)/ NP , as subpopulations with less than 4 individuals are removed after clustering.

Upon a detailed examination of Table 11, several trends and key findings emerge. In both methods, we observe a

decreasing trend in the number of subpopulations and an increasing trend in the size and proportion of the largest subpopulation as the dimensionality increases. This suggests that in high-dimensional data, the largest subpopulation may encompass more data points, possibly due to the sparser data distribution, leading to more data points being assigned to the same subpopulation. Notably, the improved clustering method exhibits more consistent performance across all dimensions in terms of the three evaluation protocols. This method effectively distinguishes subpopulations in high-dimensional data, resulting in less dominant and more evenly partitioned subpopulations, thereby accurately capturing the data distribution characteristics.

5.4.4 Computational complexity analysis

In the context of niching EAs, fitness evaluation and distance measurement are integral components. For the sake of simplicity, we assume these two processes have a constant time complexity, denoted as $O(1)$. We now delve into the computational complexity analysis of the key components of MMDE in each generation. It is important to note that the subpopulation size and the number of subpopulations can change during the evolutionary process. To account for this variability, we use their expected values in our calculations. Specifically, the expected subpopulation size is represented by NP . The improved heuristic clustering method described in Algorithm 2 requires a computational effort of $O(NP^2)$ to compute distance metrics, namely d_{inter} and d_{intra} . In Algorithm 3, which involves the auxiliary movement and archive method, the most computationally intensive step is to calculate the radius of subpopulations, a process that also has a complexity of $O(NP^2)$. Algorithm 4, which adds a solution to the archive, has a complexity of $O(NP)$. Ideally, the most computationally demanding component has a complexity of $O(NP^2)$. Consequently, the overall computational complexity of MMDE is $O(NP^2)$, aligning with the complexity observed in most niching EAs.

Table 11 The comparison between the effect of the original clustering method in [22] and improved clustering method

Dimension		2	10	20	200	1000
Method						
Original clustering	Number of subpopulations	7.6	2.55	1.75	1.45	1.25
	Size of the largest subpopulation	26.95	43.1	48.25	48.9	49.4
	Proportion of the largest subpopulation (%)	36.50	81.03	93.56	95.87	98.01
Improved clustering	Number of subpopulations	11.45	10.75	11.55	10.5	8.7
	Size of the largest subpopulation	14.3	13.55	14.05	17.35	20.6
	Proportion of the largest subpopulation (%)	15.04	17.07	17.71	23.61	31.02

6 Conclusion

This work develops an innovative neural network ensemble approach (MMDE-ELM-NNE) for simultaneously optimizing the structures and weights of multiple ELM models. The enhanced MMDE combined with ELM optimizes network structure and connection parameters, resulting in multiple compact base NNs for NNE to improve generalization performance.

To verify the effectiveness of the proposed MMDE on multimodal optimization problems, the experiments on twenty multimodal optimization problems are carried out and compared with eleven existing multimodal optimization algorithms. Experimental results show the effectiveness of the MMDE. Moreover, several experiments evaluate the feasibility and effectiveness of MMDE-ELM-NNE, demonstrating its superiority over current metaheuristic algorithms across thirteen classification datasets. A Friedman test on their testing accuracy verifies the statistical significance of differences among all algorithms, and several sensitivity analyses validate the effectiveness of the proposed algorithm. Overall, the experimental results showcase that MMDE-ELM-NNE achieves competitive performance with high classification accuracy.

Future research directions include exploring sparse structures for SLFNs and deep architectures for NNs. Additionally, investigating the combination of multimodal multi-objective optimization algorithms proficient in solving classification problems on high-dimensional data holds promise for further advancements in the field.

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s12293-025-00461-7>.

Author Contributions The authors contributed equally to each part of this paper.

Funding This work was supported in part by the Natural Science Basic Research Program of Shaanxi Province (Grant Nos. 2022JM-372 and 2022JQ-670) and by the National Natural Science Foundation of China (Grant Nos. 61966030, 62276202, and 62106186).

Data availability The data supporting the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare that there is no conflict of interest regarding the publication of this paper.

Informed consent Informed consent was obtained from all participants included in the study.

References

- Ahmadi MA, Zendehboudi S, Lohi A et al (2013) Reservoir permeability prediction by neural networks combined with hybrid genetic algorithm and particle swarm optimization. *Geophys Prospect* 61(3):582–598
- Albadra MAA, Tiuna S (2017) Extreme learning machine: a review. *Int J Appl Eng Res* 12(14):4610–4623
- Almeida LM, Ludermitr TB (2010) A multi-objective memetic and hybrid methodology for optimizing the parameters and performance of artificial neural networks. *Neurocomputing* 73(7–9):1438–1450
- Asuncion A, Newman D (2007) UCI machine learning repository. Available: <http://www.ics.uci.edu/ml/learn.MLRepository.html>
- Biswas S, Kundu S, Das S (2014) Inducing niching behavior in differential evolution through local information sharing. *IEEE Trans Evol Comput* 19(2):246–263
- Brest J, Greiner S, Boskovic B et al (2006) Self-adapting control parameters in differential evolution: a comparative study on numerical benchmark problems. *IEEE Trans Evol Comput* 10(6):646–657
- Chen H, Yao X (2010) Multiobjective neural network ensembles based on regularized negative correlation learning. *IEEE Trans Knowl Data Eng* 22(12):1738–1751
- Da Y, Xiurun G (2005) An improved PSO-based ANN with simulated annealing technique. *Neurocomputing* 63:527–533
- Dilettoso E, Salerno N (2006) A self-adaptive niching genetic algorithm for multimodal optimization of electromagnetic devices. *IEEE Trans Magn* 42(4):1203–1206
- Donate JP, Li X, Sánchez GG et al (2013) Time series forecasting by evolving artificial neural networks with genetic algorithms, differential evolution and estimation of distribution algorithm. *Neural Comput Appl* 22:11–20
- Epitropakis MG, Li X, Burke EK (2013) A dynamic archive niching differential evolution algorithm for multimodal optimization. In: 2013 IEEE congress on evolutionary computation. IEEE, pp 79–86
- Faris H, Mirjalili S, Aljarah I (2019) Automatic selection of hidden neurons and weights in neural networks using grey wolf optimizer based on a hybrid encoding scheme. *Int J Mach Learn Cybern* 10(10):2901–2920
- Friedman M (1939) A correction: the use of ranks to avoid the assumption of normality implicit in the analysis of variance. *J Am Stat Assoc* 34:109
- Gao W, Yen GG, Liu S (2013) A cluster-based differential evolution with self-adaptive strategy for multimodal optimization. *IEEE Trans Cybern*. 44(8):1314–1327
- Gionis A, Indyk P, Motwani R (1999) Similarity search in high dimensions via hashing. In: Proceedings of the 25th international conference on very large data bases. Morgan Kaufmann Publishers Inc., San Francisco, pp 518–529
- Golberg DE (1989) Genetic algorithms in search, optimization, and machine learning. Addison Wesley 1989(102):36
- Hakli H, Uğuz H (2014) A novel particle swarm optimization algorithm with Levy flight. *Appl Soft Comput* 23:333–345. <https://doi.org/10.1016/j.asoc.2014.06.034>
- Holland JH (1992) Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence. MIT Press, Cambridge
- Huang GB, Zhu QY, Siew CK (2004) Extreme learning machine: a new learning scheme of feedforward neural networks. In: 2004 IEEE international joint conference on neural networks (IEEE Cat. No. 04CH37541), IEEE, pp 985–990
- Huang GB, Chen L, Siew CK et al (2006) Universal approximation using incremental constructive feedforward networks with random hidden nodes. *IEEE Trans Neural Netw* 17(4):879–892

21. Huang GB, Zhu QY, Siew CK (2006) Extreme learning machine: theory and applications. *Neurocomputing* 70(1–3):489–501
22. Huang T, Duan DT, Gong YJ et al (2020) Concurrent optimization of multiple base learners in neural network ensembles: an adaptive niching differential evolution approach. *Neurocomputing* 396:24–38. <https://doi.org/10.1016/j.neucom.2020.02.020>
23. Hui S, Suganthan PN (2015) Ensemble and arithmetic recombination-based speciation differential evolution for multimodal optimization. *IEEE Trans Cybern* 46(1):64–74
24. Jensi R, Jiji GW (2016) An enhanced particle swarm optimization with levy flight for global optimization. *Appl Soft Comput* 43:248–261. <https://doi.org/10.1016/j.asoc.2016.02.018>
25. Jiang Y, Zhan ZH, Tan KC et al (2023) Optimizing niche center for multimodal optimization problems. *IEEE Trans Cybern* 53(4):2544–2557. <https://doi.org/10.1109/TCYB.2021.3125362>
26. Jingqiao Zhang, Sanderson A (2009) JADE: adaptive differential evolution with optional external archive. *IEEE Trans Evolut Comput* 13(5):945–958. <https://doi.org/10.1109/TEVC.2009.2014613>
27. Karaboga D, Gorkemli B, Ozturk C et al (2014) A comprehensive survey: artificial bee colony (abc) algorithm and applications. *Artif Intell Rev* 42:21–57
28. Karaboga D, et al (2005) An idea based on honey bee swarm for numerical optimization. Technical report-tr06, Erciyes university, engineering faculty, computer
29. Kennedy J, Eberhart R (1995) Particle swarm optimization. In: *Proceedings of ICNN'95-international conference on neural networks*, IEEE, pp 1942–1948
30. Kingma DP (2014) Adam: a method for stochastic optimization. *arXiv preprint arXiv:1412.6980*
31. Li JY, Chow TW, Yu YL (1995) The estimation theory and optimization algorithm for the number of hidden units in the higher-order feedforward neural network. In: *Proceedings of ICNN'95-international conference on neural networks*, IEEE, pp 1229–1233
32. Li X (2005) Efficient differential evolution using speciation for multimodal function optimization. In: *Proceedings of the 7th annual conference on Genetic and evolutionary computation*, pp 873–880
33. Li X, Engelbrecht A, Epitropakis MG (2013) Benchmark functions for cec'2013 special session and competition on niching methods for multimodal function optimization. RMIT University, Evolutionary Computation and Machine Learning Group, Australia, Technical Report
34. Liu J, Lampinen J (2005) A fuzzy adaptive differential evolution algorithm. *Soft Comput* 9:448–462
35. Mengshoel OJ, Goldberg DE (2008) The crowding approach to niching in genetic algorithms. *Evol Comput* 16(3):315–354
36. Mirjalili S (2016) Dragonfly algorithm: a new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems. *Neural Comput Appl* 27:1053–1073
37. Mirjalili S (2016) Sca: a sine cosine algorithm for solving optimization problems. *Knowl-Based Syst* 96:120–133
38. Mirjalili S, Hashim SZM, Sardroudi HM (2012) Training feed-forward neural networks using hybrid particle swarm optimization and gravitational search algorithm. *Appl Math Comput* 218(22):11125–11137
39. Ojha VK, Abraham A, Snášel V (2017) Ensemble of heterogeneous flexible neural trees using multi objective genetic programming. *Appl Soft Comput* 52:909–924
40. Ojha VK, Abraham A, Snášel V (2017) Metaheuristic design of feedforward neural networks: a review of two decades of research. *Eng Appl Artif Intell* 60:97–116
41. Pérowski A (1996) A clearing procedure as a niching method for genetic algorithms. In: *Proceedings of IEEE international conference on evolutionary computation*, IEEE, pp 798–803
42. Pettersson F, Chakraborti N, Saxén H (2007) A genetic algorithms based multi-objective neural net applied to noisy blast furnace data. *Appl Soft Comput* 7(1):387–397
43. Qin AK, Huang VL, Suganthan PN (2008) Differential evolution algorithm with strategy adaptation for global numerical optimization. *IEEE Trans Evol Comput* 13(2):398–417
44. Qu BY, Suganthan PN, Liang JJ (2012) Differential evolution with neighborhood mutation for multimodal optimization. *IEEE Trans Evol Comput* 16(5):601–614
45. Sheng M, Chen S, Liu W et al (2022) A differential evolution with adaptive neighborhood mutation and local search for multi-modal optimization. *Neurocomputing* 489:309–322
46. Storn R, Price K (1996) Minimizing the real functions of the icec'96 contest by differential evolution. In: *Proceedings of IEEE international conference on evolutionary computation*, IEEE, pp 842–844
47. Storn R, Price K (1997) Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *J Global Optim* 11:341–359
48. Tarkhaneh O, Shen H (2019) Training of feedforward neural networks for data classification using hybrid particle swarm optimization, Mantegna Lévy flight and neighborhood search. *Heliyon* 5(4):e01275. <https://doi.org/10.1016/j.heliyon.2019.e01275>
49. Thomsen R (2004) Multimodal optimization using crowding-based differential evolution. In: *Proceedings of the 2004 congress on evolutionary computation (IEEE Cat. No. 04TH8753)*, IEEE, pp 1382–1389
50. Tieleman T (2012) Lecture 6.5-rmsprop: aivide the gradient by a running average of its recent magnitude. *COURSERA Neural Netw Mach Learn* 4(2):26
51. Wang X, Huang Y (2011) Convergence study in extended kalman filter-based training of recurrent neural networks. *IEEE Trans Neural Netw* 22(4):588–600
52. Wang ZJ, Zhan ZH, Lin Y et al (2019) Automatic niching differential evolution with contour prediction approach for multimodal optimization problems. *IEEE Trans Evol Comput* 24(1):114–128
53. Wang ZJ, Zhou YR, Zhang J (2020) Adaptive estimation distribution distributed differential evolution for multimodal optimization problems. *IEEE Trans Cybern* 52(7):6059–6070
54. Wang ZJ, Zhan ZH, Li Y et al (2023) Fitness and distance based local search with adaptive differential evolution for multimodal optimization problems. *IEEE Trans Emerg Top Comput Intell* 7(3):684–699. <https://doi.org/10.1109/TETCI.2023.3234575>
55. Wei Z, Gao W, Li G et al (2022) A penalty-based differential evolution for multimodal optimization. *IEEE Trans Cybern* 52(7):6024–6033. <https://doi.org/10.1109/TCYB.2021.3117359>
56. Yang Q, Chen WN, Yu Z et al (2016) Adaptive multimodal continuous ant colony optimization. *IEEE Trans Evol Comput* 21(2):191–205
57. Yang XS (2009) Firefly algorithms for multimodal optimization. In: *International symposium on stochastic algorithms*. Springer, Berlin, pp 169–178
58. Yin X, Gernay N (1993) A fast genetic algorithm with sharing scheme using cluster analysis methods in multimodal function optimization. In: *Artificial neural nets and genetic algorithms: proceedings of the international conference in Innsbruck, Austria, 1993*. Springer, Berlin, pp 450–457
59. Zeng X, Yeung DS (2006) Hidden neuron pruning of multilayer perceptrons using a quantified sensitivity measure. *Neurocomputing* 69(7–9):825–837
60. Zhang J, Morris AJ (1998) A sequential learning approach for single hidden layer neural networks. *Neural Netw* 11(1):65–80
61. Zhang J, Chen D, Yang Q et al (2023) Proximity ranking-based multimodal differential evolution. *Swarm Evol Comput* 78:101277
62. Zhang YH, Gong YJ, Zhang HX et al (2016) Toward fast niching evolutionary algorithms: a locality sensitive hashing-based approach. *IEEE Trans Evol Comput* 21(3):347–362

63. Zhang YH, Gong YJ, Gao Y et al (2019) Parameter-free voronoi neighborhood for evolutionary multimodal optimization. *IEEE Trans Evol Comput* 24(2):335–349
64. Zhao H, Zhan ZH, Lin Y et al (2019) Local binary pattern-based adaptive differential evolution for multimodal optimization problems. *IEEE Trans cybern* 50(7):3343–3357

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.